Nordic Model analysis: TALIS 2024

Blinded

2026-10-12

Setup

Load data and create training and test data

Create training and test data

```
set.seed(1312)

CountrySizes <- BTGINTT4 %>%
  count(CNTRY) %>%
  mutate(Nover=n-4000) %>%
  mutate( # select all over 4000 or 10%, whichever is bigger to be test data. this limits tra
ining data size for large countries, which was suggested by a reviewer
    Ntrain=ifelse(Nover/n <= 0.1, as.integer(n*0.1) , Nover)) %>%
  select(-Nover,-n)

BTGINTT4 %>%
  count(CNTRY,IDSCHOOL) %>%
  merge(CountrySizes) %>%
  mutate(train=0) %>%
  group_by(CNTRY) %>%
  group_modify(~{
    while ( sum(.x$train) < .x$Ntrain[1] ) {
      a <- sample(1:nrow(.x),1)
      .x[a, "train"] <- .x[a,"n"]
    }
    .x
  } ) %>%
  as.data.frame %>%
  filter(train>0) %>%
  with( paste.(CNTRY,IDSCHOOL) ) ->
  TestSchools

Test <- BTGINTT4 %>% filter(paste.(CNTRY,IDSCHOOL) %in% TestSchools )
BTGINTT4 <- BTGINTT4 %>% filter(!( paste.(CNTRY,IDSCHOOL) %in% TestSchools ) )

cat("\nNrows of test data: ", nrow(Test))
```

```
##
## Nrows of test data:  30515
```

```
cat("\nNrows of training data: ", nrow(BTGINTT4))
```

```
##
## Nrows of training data:  158831
```

```
cat("\nPercentage of data in test data: ", nrow(Test)/(nrow(Test)+nrow(BTGINTT4)) )
```

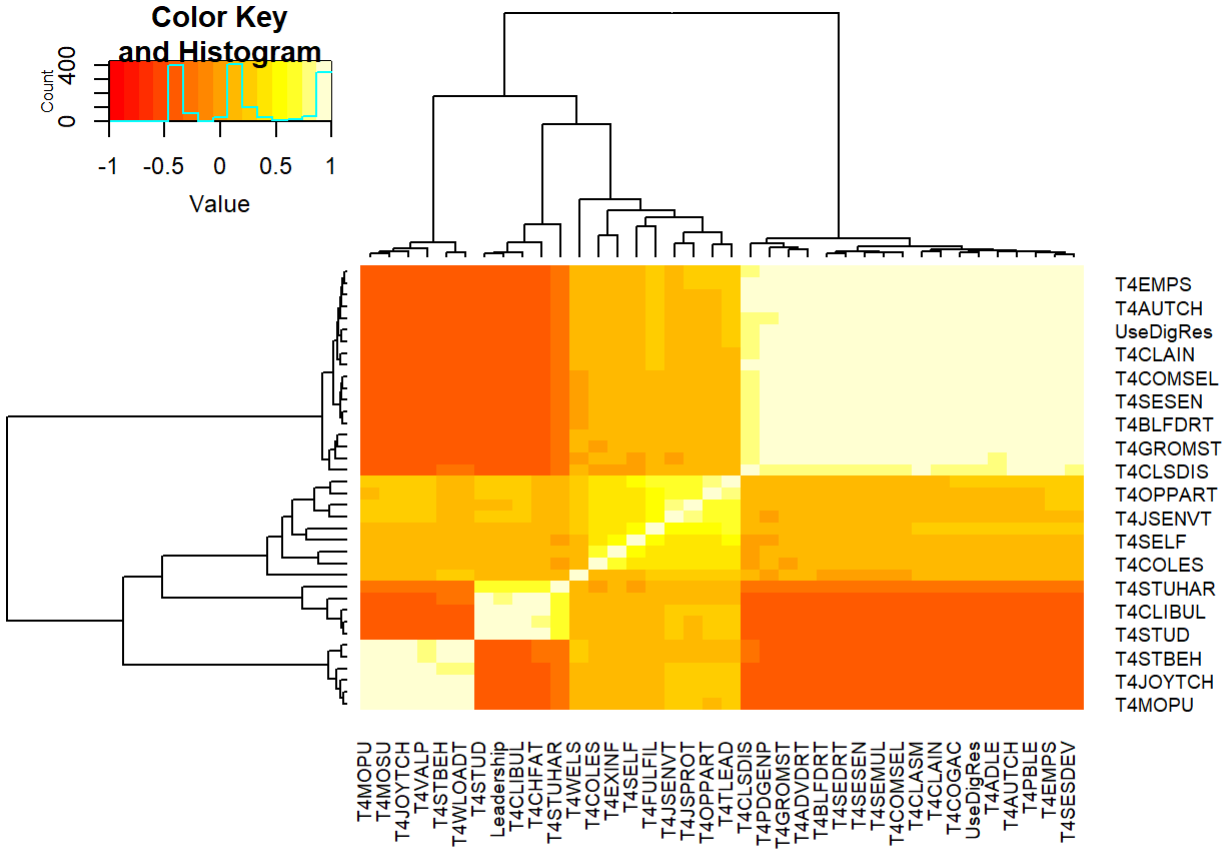
```
##
## Percentage of data in test data:  0.16116
```

Test for high correlations and remove those to reduce multicollinearity issues

There are some scales that are correlated over 0.79. We combine these scales to ensure that high collinearity does not impact our ability to explore variable importance.

Scale1	Scale2	Correlation
T4DRTIIA	T4DRTWCI	0.8067
T4INSTLE	T4RELEAD	0.7988

```
BTGINTT4 %>% select(T4EXINF,T4COLES,T4SELF,T4SEMUL,T4GROMST,T4SESEN,T4COMSEL,T4SEDRT,T4BLFDR
T,T4ADVDRT,T4CLAIN,T4COGAC,T4CLASM,T4PDGENP,T4EMPS,T4CLSDIS,T4ADLE,T4PBLE,T4AUTCH,T4FULFIL,T4
OPPART,T4SESDEV,T4TLEAD,T4STUD,T4CLIBUL,T4STUHAR,T4CHFAT,T4MOPU,T4MOSU,T4WELS,T4WLOADT,T4STBE
H,T4JSENVT,T4JSPROT,T4VALP,T4JOYTCH,UseDigRes,Leadership) %>% cor(use="pair") %>% gplots::hea
tmap.2(trace="none",cexCol=.7,cexRow=.7,margins=c(8,8), dendrogram="both" )
```



Test for ICCs at the school and country level

```
iccs <- misty::multilevel.icc(BTGINTT4 %>% select(T4EXINF,T4COLES,T4SELF,T4SEMUL,T4GROMST,T4S
ESEN,T4COMSEL,T4SEDRT,T4BLFDRT,T4ADVDRT,T4CLAIN,T4COGAC,T4CLASM,T4PDGENP,T4EMPS,T4CLSDIS,T4AD
LE,T4PBLE,T4AUTCH,T4FULFIL,T4OPPART,T4SESDEV,T4TLEAD,T4STUD,T4CLIBUL,T4STUHAR,T4CHFAT,T4MOPU,
T4MOSU,T4WELS,T4WLOADT,T4STBEH,T4JSENVT,T4JSPROT,T4VALP,T4JOYTCH,CNTRY,UseDigRes,Leadership,I
DSCHOOL) %>% mutate_all(function(x) ifelse(x==0,NA,x)) ,
                      cluster = c( "CNTRY" , "IDSCHOOL" ),
                      type="1a")
iccs %>% t %>% knitr::kable(caption="Table of percentages of variation in variables that is b
etween schools",digits=2)
```

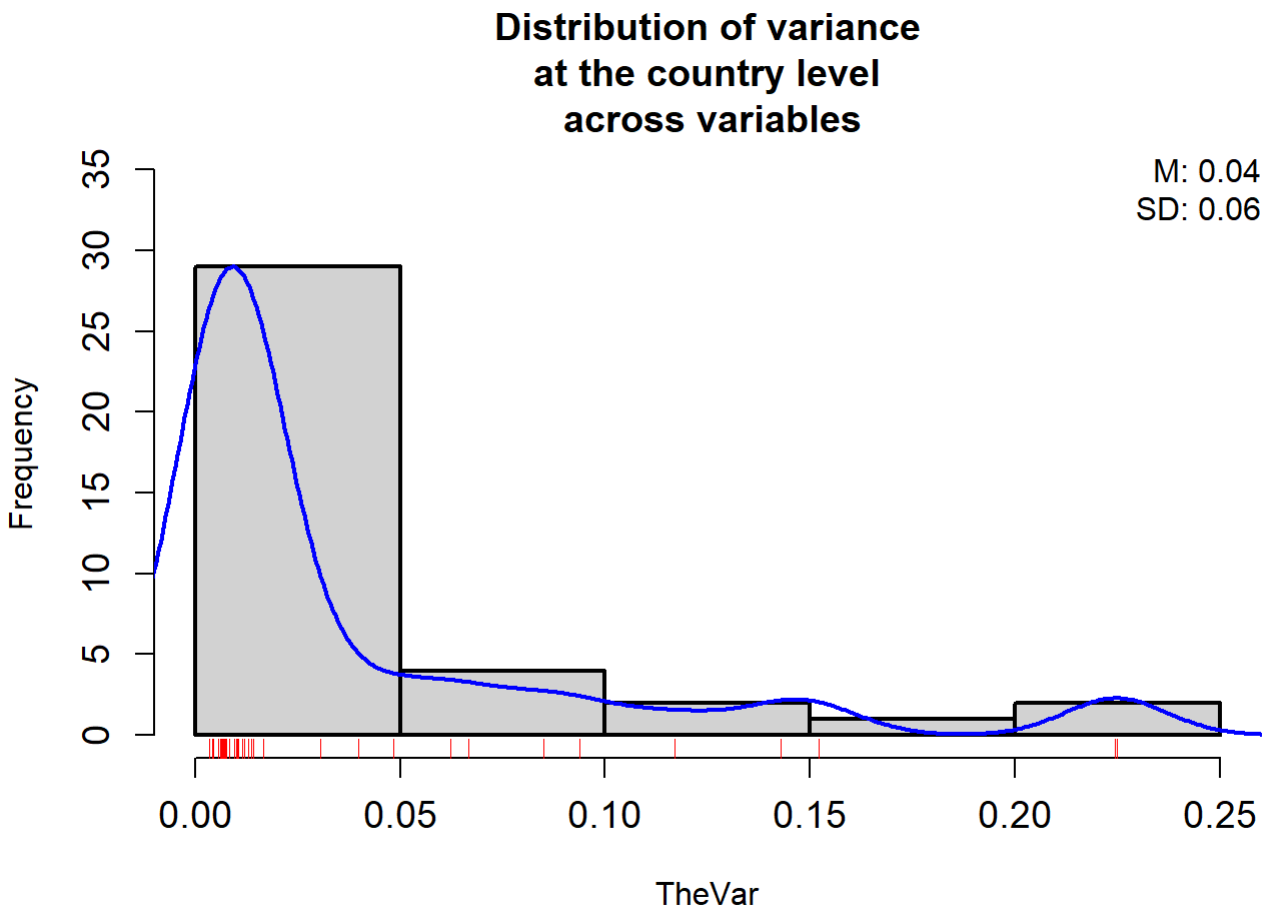
Table of percentages of variation in variables that is between schools

	CNTRY	IDSCHOOL
T4EXINF	0.06	0.05
T4COLES	0.15	0.06
T4SELF	0.14	0.03
T4SEMUL	0.01	0.00
T4GROMST	0.01	0.00
T4SESEN	0.01	0.00
T4COMSEL	0.01	0.00
T4SEDRT	0.01	0.00
T4BLFDRT	0.01	0.00
T4ADVDRT	0.00	0.00
T4CLAIN	0.01	0.00
T4COGAC	0.01	0.00
T4CLASM	0.01	0.00
T4PDGENP	0.01	0.00
T4EMPS	0.01	0.00
T4CLSDIS	0.01	0.00
T4ADLE	0.04	0.00
T4PBLE	0.01	0.00
T4AUTCH	0.01	0.00
T4FULFIL	0.12	0.03
T4OPPART	0.09	0.07
T4SESDEV	0.01	0.00
T4TLEAD	0.08	0.07

	CNTRY	IDSCHOOL
T4STUD	0.00	0.00
T4CLIBUL	0.01	0.00
T4STUHAR	0.22	0.00
T4CHFAT	0.00	0.00
T4MOPU	0.01	0.00
T4MOSU	0.01	0.00
T4WELS	0.22	0.05
T4WLOADT	0.01	0.00
T4STBEH	0.01	0.00
T4JSENVT	0.05	0.08
T4JSPROT	0.07	0.05
T4VALP	0.03	0.00
T4JOYTCH	0.01	0.00
UseDigRes	0.02	0.00
Leadership	0.01	0.00

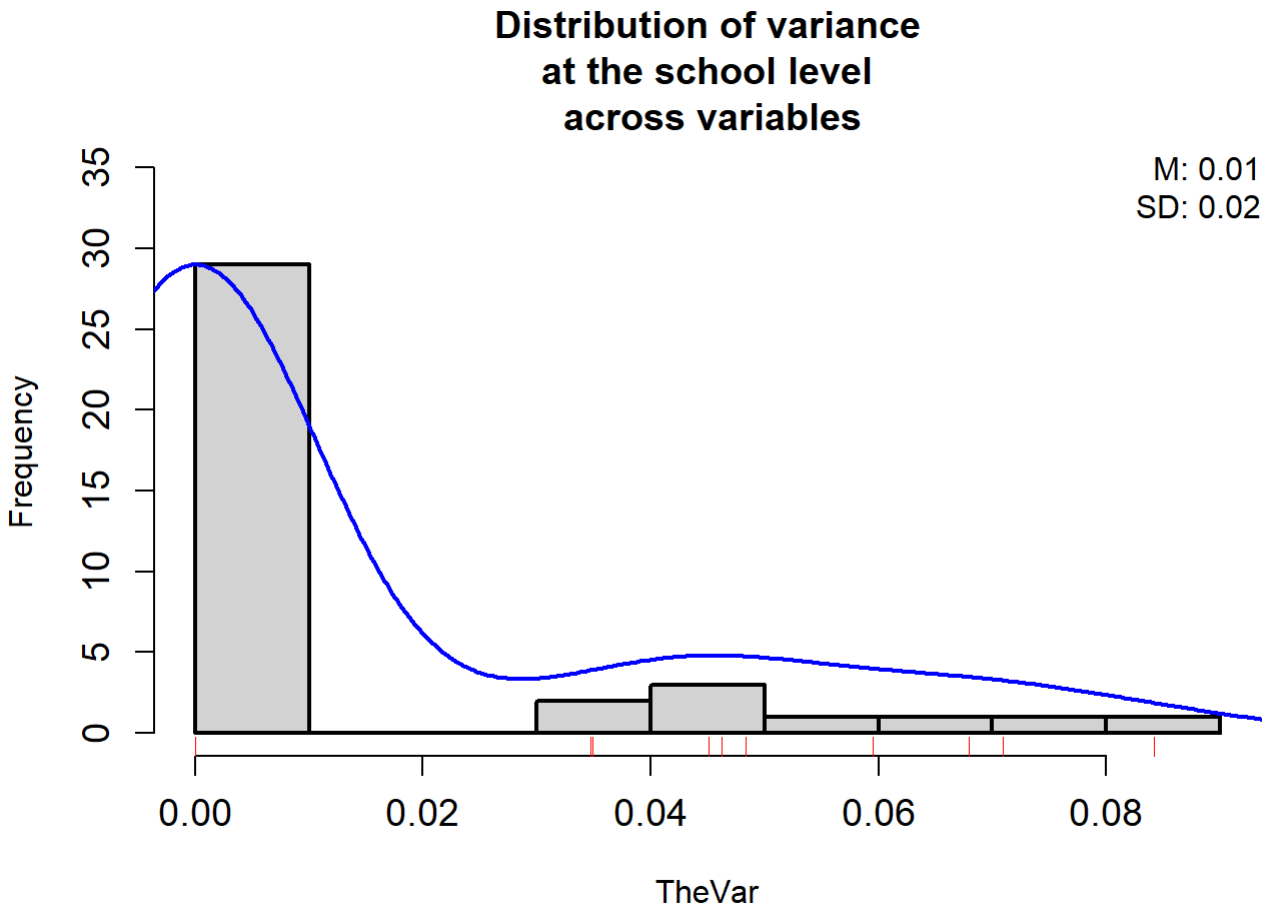
```
hist.rug(iccs["CNTRY",],main="Distribution of variance\nat the country level\n across variabl
es")
```

```
##    vars  n mean   sd median trimmed  mad min  max range skew kurtosis   se
## X1    1 38 0.04 0.06   0.01    0.03 0.01   0 0.22  0.22 1.92     2.78 0.01
```




```
hist.rug(iccs["IDSCHOOL",],main="Distribution of variance\nat the school level\n across varia
bles")
```

```
##    vars  n mean   sd median trimmed mad min  max range skew kurtosis se
## X1    1 38 0.01 0.02      0    0.01   0   0 0.08  0.08 1.54     0.87  0
```



BART Model

run model

The CV model to verify ideal parameters is not run due to the amount of time it takes to run.

```
model_cv <- xbart(Nordic~ T4EXINF+T4COLES+T4SELF+T4SEMUL+T4GROMST+T4SESEN+T4COMSEL+T4SEDRT+T4
BLFDRT+T4ADVDRT+T4CLAIN+T4COGAC+T4CLASM+T4PDGENP+T4EMPS+T4CLSDIS+T4ADLE+T4PBLE+T4AUTCH+T4FULF
IL+T4OPPART+T4SESDEV+T4TLEAD+T4STUD+T4CLIBUL+T4STUHAR+T4CHFAT+T4MOPU+T4MOSU+T4WELS+T4WLOADT+T
4STBEH+T4JSENVT+T4JSPROT+T4VALP+T4JOYTCH+UseDigRes+Leadership,
  data = BTGINTT4,n.burn = c(20000,1000,1000), n.samples=2000,
  method="k-fold",n.test=5,n.reps=10,
  n.trees = c(50,100,200),
  k=c(1.5, 2.0),
  power=c(1.5,2),
  base=c(0.95,0.99),
  loss = "log",
  n.threads = 20,
  seed=1121
)
save(model_cv, file="crossvalidation.rData")
apply(model_cv,c(2,3,4),mean)
apply(model_cv,c(2,3,4,5),mean) %>% as.data.frame %>% rownames_to_column(var="Ntree") %>% gat
her(var,val,-Ntree) %>% arrange(-val) %>% head(5)
```

```
X = BTGINTT4 %>% select(T4EXINF,T4COLES,T4SELF,T4SEMUL,T4GROMST,T4SESEN,T4COMSEL,T4SEDRT,T4BL
FDRT,T4ADVDRT,T4CLAIN,T4COGAC,T4CLASM,T4PDGENP,T4EMPS,T4CLSDIS,T4ADLE,T4PBLE,T4AUTCH,T4FULFI
L,T4OPPART,T4SESDEV,T4TLEAD,T4STUD,T4CLIBUL,T4STUHAR,T4CHFAT,T4MOPU,T4MOSU,T4WELS,T4WLOADT,T4
STBEH,T4JSENVT,T4JSPROT,T4VALP,T4JOYTCH,UseDigRes,Leadership) %>% as.matrix

model <- bart(
  x.train = as.matrix(X),
  y.train = factor(BTGINTT4$Nordic),
  ntree = 50, power=2, base=0.99, k=1.5,
  nskip = 70000, ndpost = 40000, keepevery = 100,
  nchain=8, nthread=8,
  seed=561,
  keeptrees = TRUE
)
```

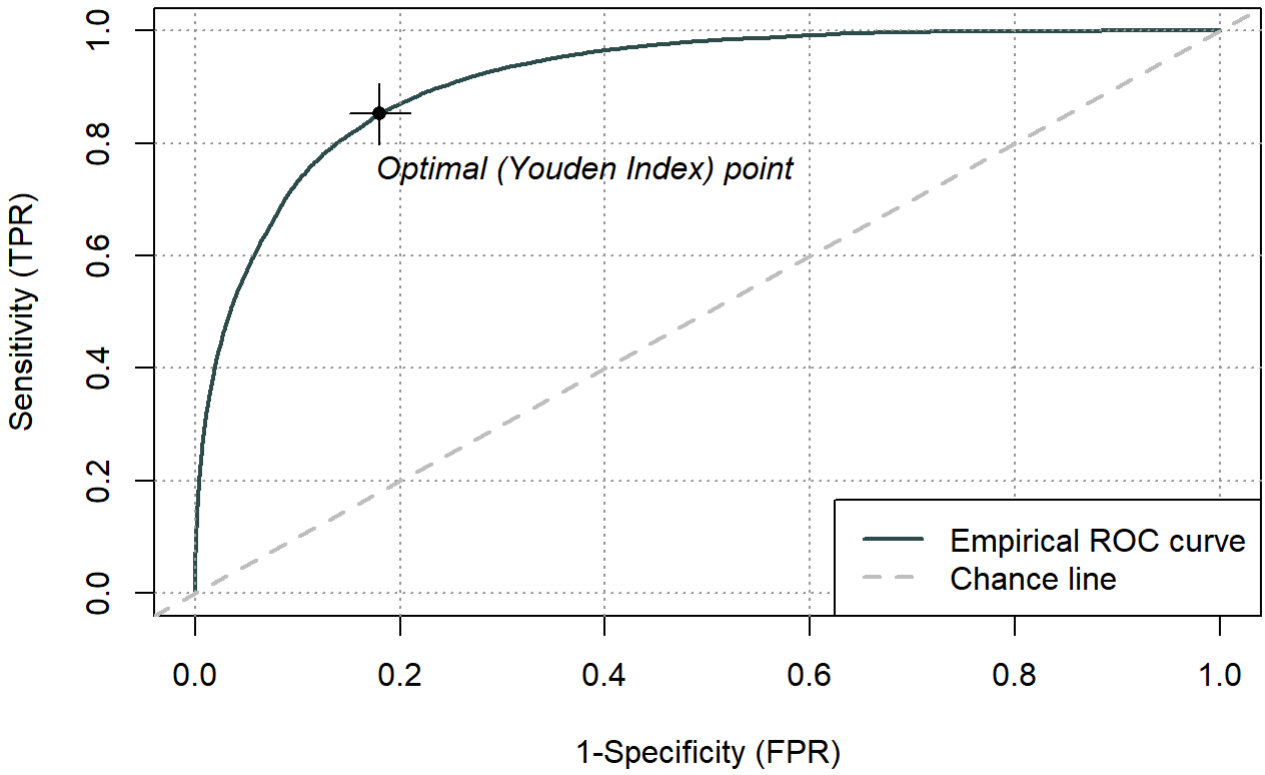
```
##
## Running BART with binary y
##
## number of trees: 50
## number of chains: 8, default number of threads 8
## tree thinning rate: 100
## Prior:
##  k prior fixed to 1.500000
##  power and base for tree prior: 2.000000 0.990000
##  use quantiles for rule cut points: false
##  proposal probabilities: birth/death 0.50, swap 0.10, change 0.40; birth 0.50
## data:
##  number of training observations: 158831
##  number of test observations: 0
##  number of explanatory variables: 38
##
## Cutoff rules c in x<=c vs x>c
## Number of cutoffs: (var: number of possible c):
## (1: 100) (2: 100) (3: 100) (4: 100) (5: 100)
## (6: 100) (7: 100) (8: 100) (9: 100) (10: 100)
## (11: 100) (12: 100) (13: 100) (14: 100) (15: 100)
## (16: 100) (17: 100) (18: 100) (19: 100) (20: 100)
## (21: 100) (22: 100) (23: 100) (24: 100) (25: 100)
## (26: 100) (27: 100) (28: 100) (29: 100) (30: 100)
## (31: 100) (32: 100) (33: 100) (34: 100) (35: 100)
## (36: 100) (37: 100) (38: 100)
## offsets:
##  reg : 0.00 0.00 0.00 0.00 0.00
## Running mcmc loop:
## [7] iteration: 100 (of 40000)
## [1] iteration: 100 (of 40000)
## [4] iteration: 100 (of 40000)
## [6] iteration: 100 (of 40000)
## [5] iteration: 100 (of 40000)
## [3] iteration: 100 (of 40000)
## [8] iteration: 100 (of 40000)
## [2] iteration: 100 (of 40000)
## [7] iteration: 200 (of 40000)
## [1] iteration: 200 (of 40000)
## [4] iteration: 200 (of 40000)
## [6] iteration: 200 (of 40000)
## [5] iteration: 200 (of 40000)
...
## [2] iteration: 39500 (of 40000)
## [1] iteration: 39900 (of 40000)
## [8] iteration: 40000 (of 40000)
## [3] iteration: 39500 (of 40000)
## [4] iteration: 39900 (of 40000)
## [2] iteration: 39600 (of 40000)
## [1] iteration: 40000 (of 40000)
## [3] iteration: 39600 (of 40000)
## [4] iteration: 40000 (of 40000)
## [2] iteration: 39700 (of 40000)
## [3] iteration: 39700 (of 40000)
## [2] iteration: 39800 (of 40000)
```

```
## [3] iteration: 39800 (of 40000)
## [2] iteration: 39900 (of 40000)
## [3] iteration: 39900 (of 40000)
## [2] iteration: 40000 (of 40000)
## [3] iteration: 40000 (of 40000)
## total seconds in loop: 28666.089401
##
## Tree sizes, last iteration:
## [1] 5 5 11 6 7 7 10 5 6 7 10 10 6 10 10 7 8 10
## 12 6 6 3 12 6 9 10 6 9 6 6 10 5 7 6 8 7 4 12
## 10 6 10 4 6 5 6 6 8 5 4 7
## [2] 5 6 4 6 10 4 7 9 10 7 5 7 13 5 8 8 12 6
## 4 4 5 5 5 7 12 9 4 10 3 8 6 12 7 12 4 12 7 10
## 10 9 5 7 12 6 5 8 7 7 5 9
## [3] 10 8 9 7 5 12 5 4 9 5 7 8 9 4 9 12 6 10
## 8 5 8 7 6 10 11 4 7 5 3 9 7 8 9 6 5 10 7 7
## 11 7 7 7 10 7 9 8 8 11 8 7
## [4] 6 10 6 8 9 7 8 9 5 8 9 9 9 8 8 7 8 10
## 10 9 4 10 8 4 8 6 6 7 6 9 6 8 8 9 9 7 11 8
## 4 6 5 9 5 7 11 8 5 9 4 4
## [5] 8 9 2 8 7 7 7 6 13 11 8 8 11 7 10 7 8 3
## 8 7 13 9 6 7 6 6 8 10 7 8 8 11 8 5 7 9 5 7
## 7 11 4 8 10 6 4 5 7 11 5 7
## [6] 8 5 10 8 12 6 7 6 7 6 7 9 6 3 9 9 8 9
## 5 7 8 8 8 11 8 5 10 7 8 5 6 9 3 7 6 7 11 10
## 5 9 11 6 10 7 8 12 10 9 5 8
## [7] 7 7 6 6 7 9 13 5 7 5 11 12 7 9 6 10 7 7
## 11 15 7 8 6 6 6 10 8 10 7 9 6 6 8 8 9 8 7 6
## 6 5 6 8 6 9 8 6 7 6 4 12
## [8] 10 4 7 10 8 9 10 9 5 6 12 11 6 12 9 5 7 5
## 10 10 10 6 6 5 9 6 13 5 5 7 7 6 6 3 5 7 8 9
## 7 9 7 10 4 10 6 9 6 7 5 5
##
## Variable Usage, last iteration (var:count):
## (1: 73) (2: 75) (3: 56) (4: 52) (5: 77)
## (6: 51) (7: 58) (8: 56) (9: 73) (10: 47)
## (11: 54) (12: 57) (13: 51) (14: 48) (15: 52)
## (16: 78) (17: 47) (18: 59) (19: 75) (20: 104)
## (21: 137) (22: 54) (23: 86) (24: 109) (25: 101)
## (26: 114) (27: 55) (28: 12) (29: 29) (30: 109)
## (31: 45) (32: 67) (33: 64) (34: 103) (35: 111)
## (36: 36) (37: 75) (38: 62)
## DONE BART
```

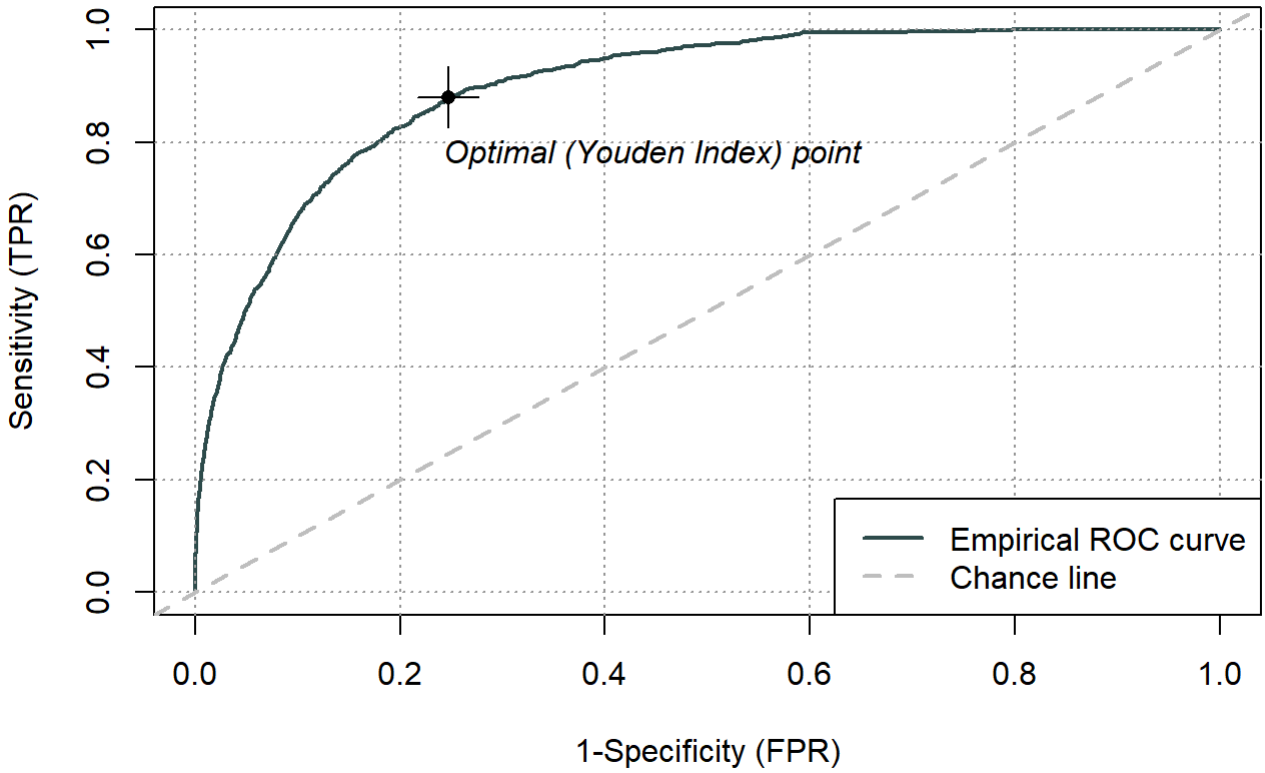
get data for ROC curves

```
predictedTrain <- predict(model, X)
predictedTest <- predict(model, Test %>% select(T4EXINF,T4COLES,T4SELF,T4SEMUL,T4GROMST,T4SES
EN,T4COMSEL,T4SEDRT,T4BLFDRT,T4ADVDRT,T4CLAIN,T4COGAC,T4CLASM,T4PDGENP,T4EMPS,T4CLSDIS,T4ADL
E,T4PBLE,T4AUTCH,T4FULFIL,T4OPPART,T4SESDEV,T4TLEAD,T4STUD,T4CLIBUL,T4STUHAR,T4CHFAT,T4MOPU,T
4MOSU,T4WELS,T4WLOADT,T4STBEH,T4JSENVT,T4JSPROT,T4VALP,T4JOYTCH,UseDigRes,Leadership)   )

ROCit_objTr <- rocit(score=apply(predictedTrain,2,mean),class=BTGINTT4$Nordic,negref =0)
ROCit_obj   <- rocit(score=apply(predictedTest ,2,mean),class=Test$Nordic    ,negref =0)
ROCIT_NordicTr <- plot(ROCit_objTr)
```




```
ROCIT_Nordic   <- plot(ROCit_obj  )
```




```
save(ROCIT_NordicTr,ROCit_objTr,
     ROCIT_Nordic  ,ROCit_obj  ,
     file="ROCIT_Nordic.rData")
```

get data for trees

```
tree_data <- extractTrees.bart.fixed(model = model, data = BTGINTT4 %>% select(T4EXINF,T4COLE
S,T4SELF,T4SEMUL,T4GROMST,T4SESEN,T4COMSEL,T4SEDRT,T4BLFDRT,T4ADVDRT,T4CLAIN,T4COGAC,T4CLASM,
T4PDGENP,T4EMPS,T4CLSDIS,T4ADLE,T4PBLE,T4AUTCH,T4FULFIL,T4OPPART,T4SESDEV,T4TLEAD,T4STUD,T4CL
IBUL,T4STUHAR,T4CHFAT,T4MOPU,T4MOSU,T4WELS,T4WLOADT,T4STBEH,T4JSENVT,T4JSPROT,T4VALP,T4JOYTC
H,UseDigRes,Leadership) )
```

```
## Extracting Observation Data...
```

```
hideHelper1 <- function(df) {
        class(df) <- c("hideHelper1", class(df))
        df }

tree_data <- hideHelper1(tree_data)
```

create shap values

```
pfun <- function (object, newdata) {
  predict(object , newdata)
}

shap_results <- Explain(model,
          X = as.data.frame(X),
          newdata = Test %>% select(T4EXINF,T4COLES,T4SELF,T4SEMUL,T4GROMST,T4SESEN,T4COMSEL,
T4SEDRT,T4BLFDRT,T4ADVDRT,T4CLAIN,T4COGAC,T4CLASM,T4PDGENP,T4EMPS,T4CLSDIS,T4ADLE,T4PBLE,T4AU
TCH,T4FULFIL,T4OPPART,T4SESDEV,T4TLEAD,T4STUD,T4CLIBUL,T4STUHAR,T4CHFAT,T4MOPU,T4MOSU,T4WELS,
T4WLOADT,T4STBEH,T4JSENVT,T4JSPROT,T4VALP,T4JOYTCH,UseDigRes,Leadership) %>% as.data.frame,
          pred_wrapper = pfun)
```

Check model convergence and calculate ESS

```
macro_predictions <- split( pnorm(model$yhat.train) %>% apply(c(1), mean) %>% t ,
                            rep(1:model$call$nchain , each=model$call$ndpost/model$call$keepe
very) )
mcmc_chains <- lapply(1:length(macro_predictions), function(i) {
  mcmc(macro_predictions[[i]]) }) %>%  as.mcmc.list

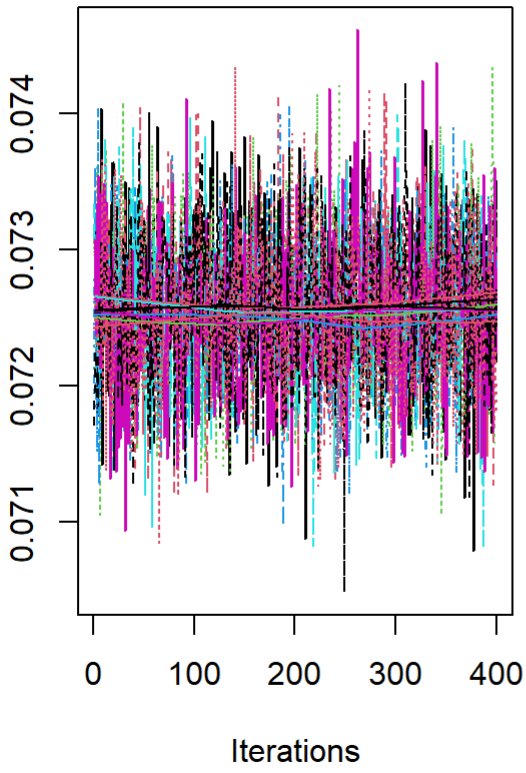
gelman.diag(mcmc_chains)
```

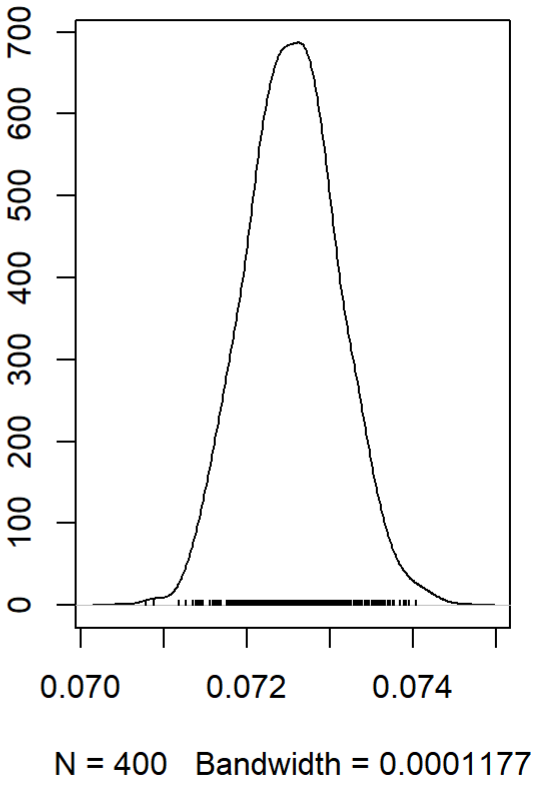
```
## Potential scale reduction factors:
##
##      Point est. Upper C.I.
## [1,]          1       1.01
```

```
print(coda::effectiveSize(mcmc_chains))
```

```
##     var1
## 3044.548
```

```
plot(mcmc_chains)
```

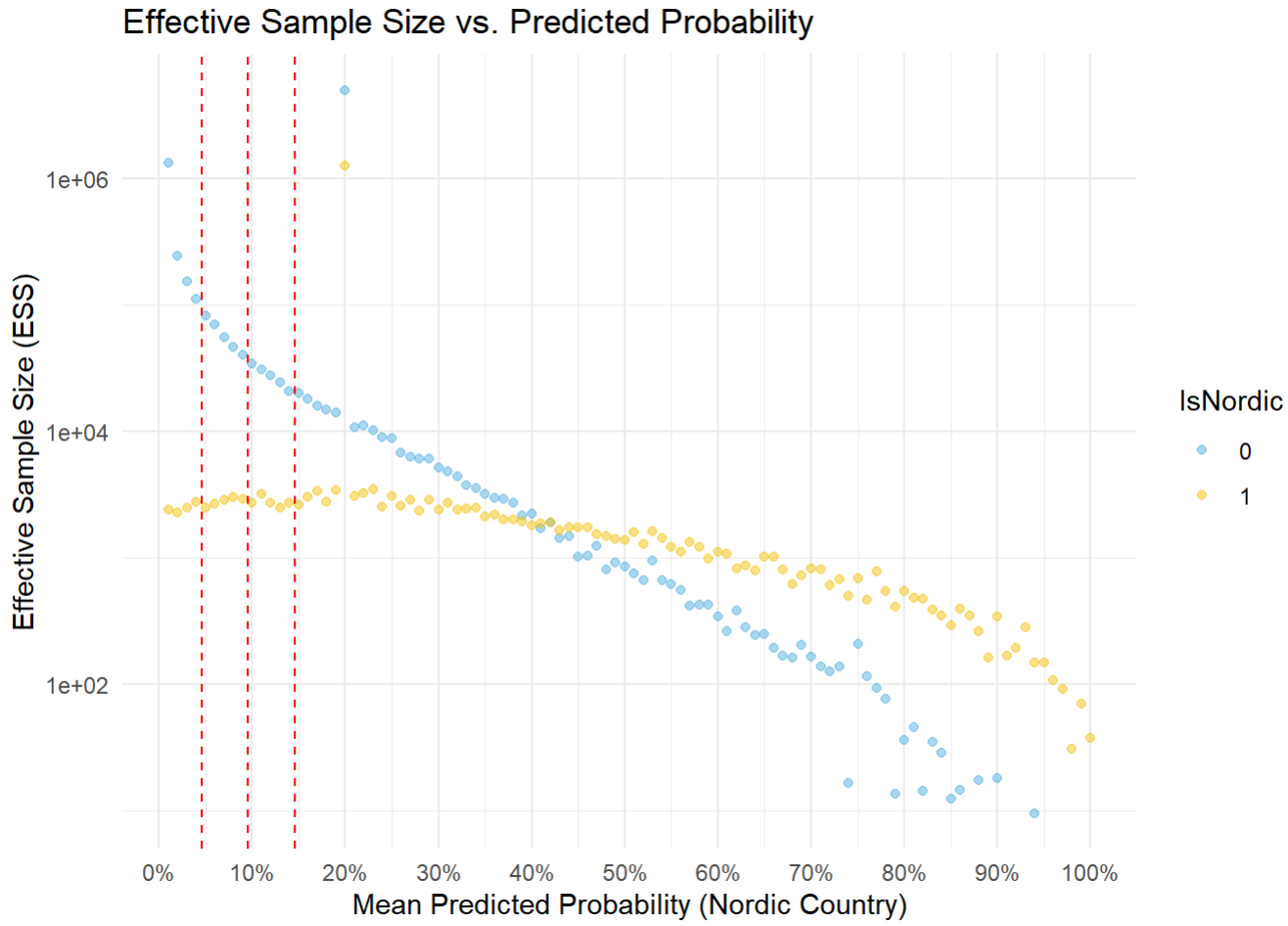

0.071
0.072
0.073
0.074
0
100
200
300
400
Iterations


0
100
200
300
400
500
600
700
0.070
0.072
0.074
N = 400 Bandwidth = 0.0001177

```
pred_prob_matrix <- model$yhat.train |> pnorm()
# Calculate ESS for each column (teacher) by looking across samples for each case
ess_values <- apply(pred_prob_matrix, 2, effectiveSize)
# Calculate the mean probability for each teacher
mean_probs <- colMeans(pred_prob_matrix)
# Create a data frame combining Teacher ID, Mean Probability, and ESS
teacher_diagnostics <- data.frame(
  IsNordic=model$fit$data@y,
  teacher_id = 1:length(mean_probs),
  mean_prob = mean_probs,
  ess = ess_values  )

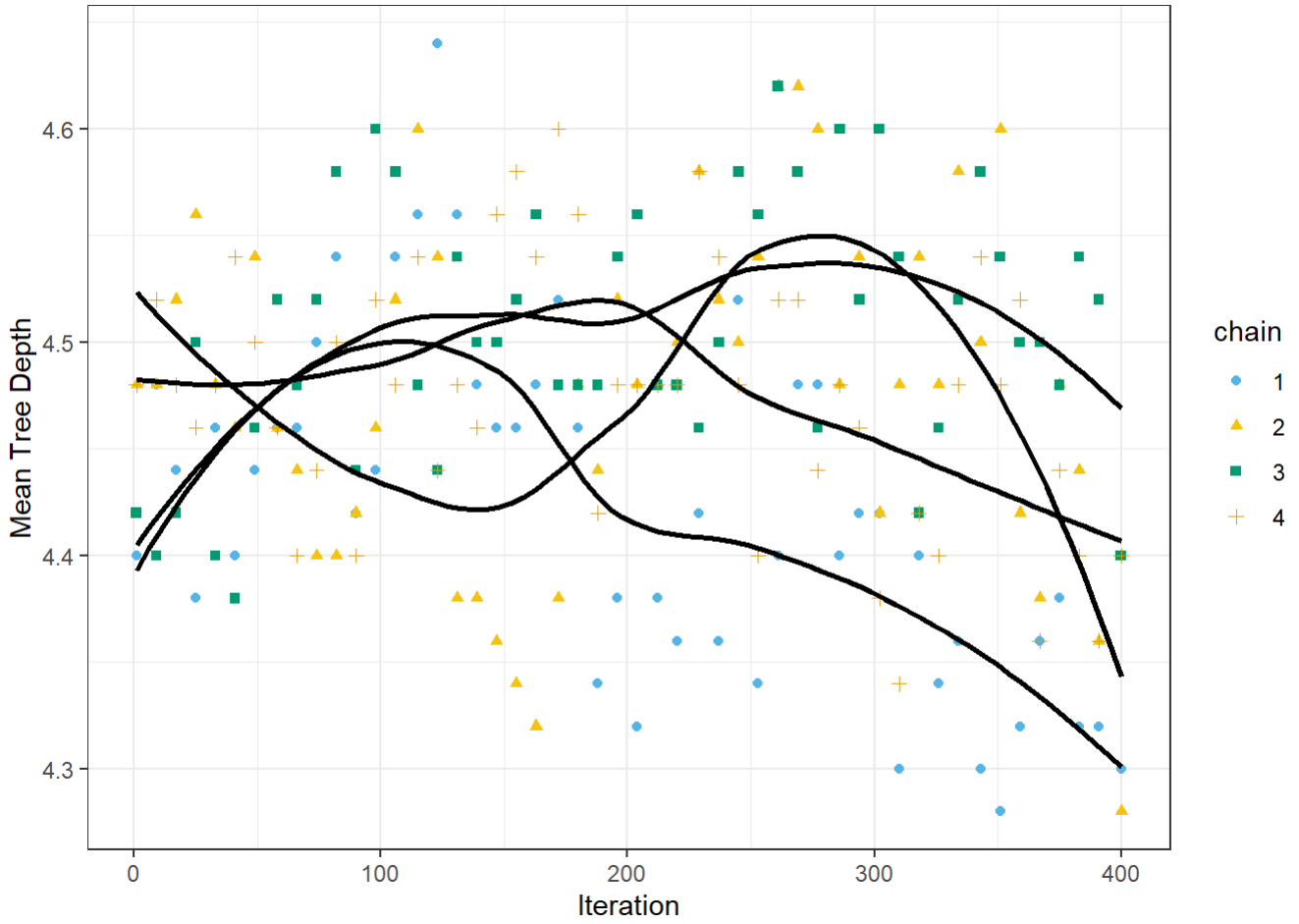
cutoff_zone <- teacher_diagnostics[teacher_diagnostics$mean_prob >= ROCIT_NordicTr$optimal[4]
-0.05 &
                                   teacher_diagnostics$mean_prob <= ROCIT_NordicTr$optimal[4]
+0.05, ]

teacher_diagnostics |>
  mutate(mean_prob=cut(mean_prob,seq(0,1,0.01),include.lowest=T) |> factor() |> as.integer()
,
         mean_prob=mean_prob/100,
         IsNordic=factor(IsNordic) ) |>
  group_by(mean_prob,IsNordic) |>
  summarize(ess=sum(ess,na.rm=T) , .groups="drop") |>
  ggplot(aes(x = mean_prob, y = ess,color=IsNordic)) +
  geom_point(alpha = 0.5) +
  geom_vline(xintercept = c(ROCIT_NordicTr$optimal[4]-0.05,
                            ROCIT_NordicTr$optimal[4],
                            ROCIT_NordicTr$optimal[4]+0.05),
             linetype = "dashed", color = "red") +
  labs(title = "Effective Sample Size vs. Predicted Probability",
       x = "Mean Predicted Probability (Nordic Country)",
       y = "Effective Sample Size (ESS)") +
  theme_minimal()+
  scale_x_continuous(breaks=seq(0,1,0.1),label=scales::percent)+
  scale_y_log10()
```

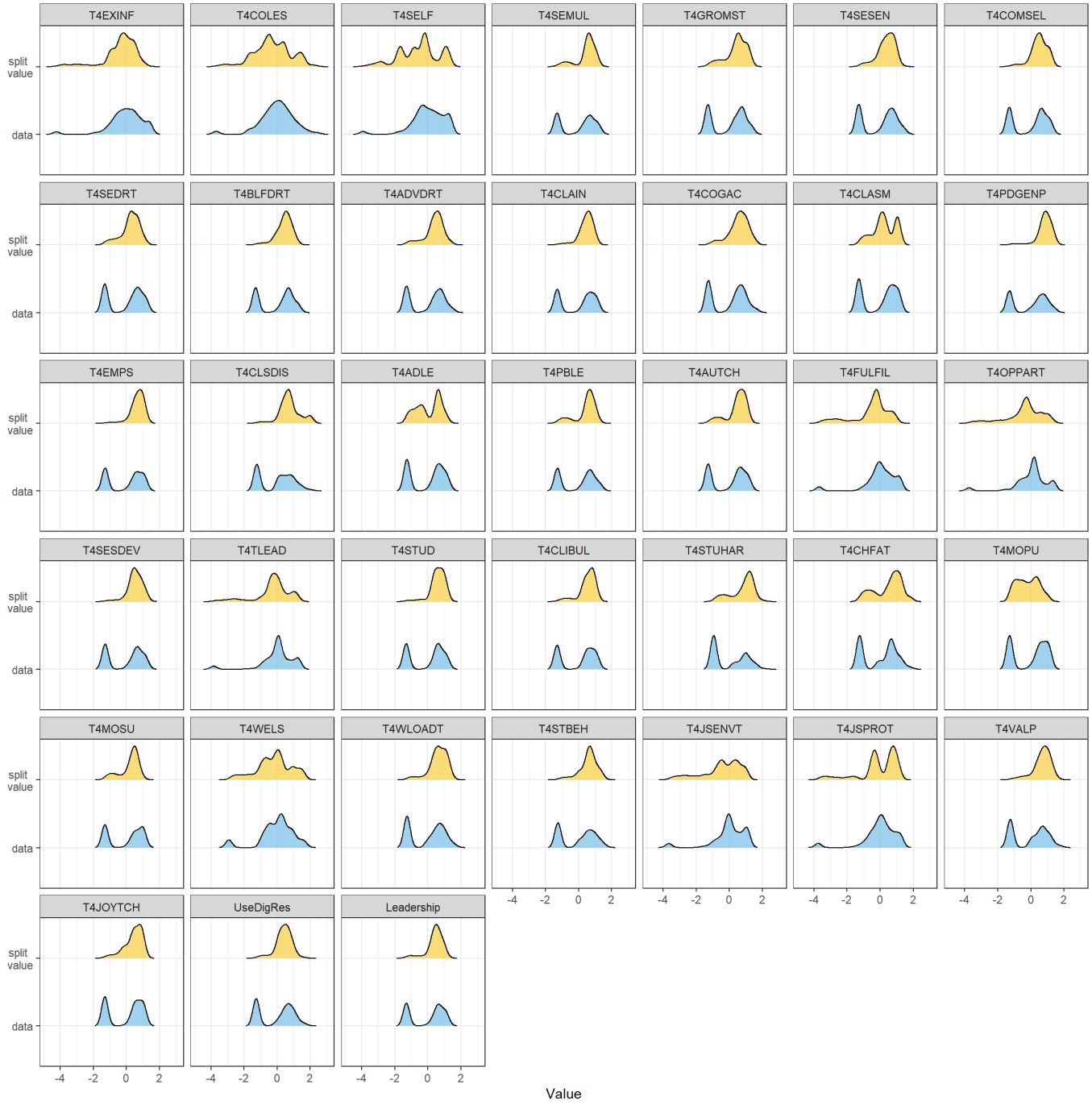


tree information

```
treeDepthf <- function (trees) {
    maxIter <- max(trees$structure$iteration)
    newTrees <- trees$structure %>%
      count(iteration, treeNum, chain, depthMax) %>%
      group_by(iteration,chain) %>%
      summarize(avgDepth = mean(depthMax),
                .groups="drop") %>%
      mutate(chain=factor(chain))
    ylimMax <- max(newTrees$avgDepth)
    p <- ggplot(newTrees, aes(iteration, avgDepth,color=chain,shape=chain)) +
      geom_point() +
      geom_smooth(formula = y ~ x, method = "loess", colour = "black", se = F) +
      theme_bw() +
      xlab("Iteration") +
      ylab("Mean Tree Depth")
    return(p)
}
treeDepthf( trees = tree_data)
```




```
splitDensity(trees = tree_data, data = BTGINTT4, display = 'dataSplit' )
```

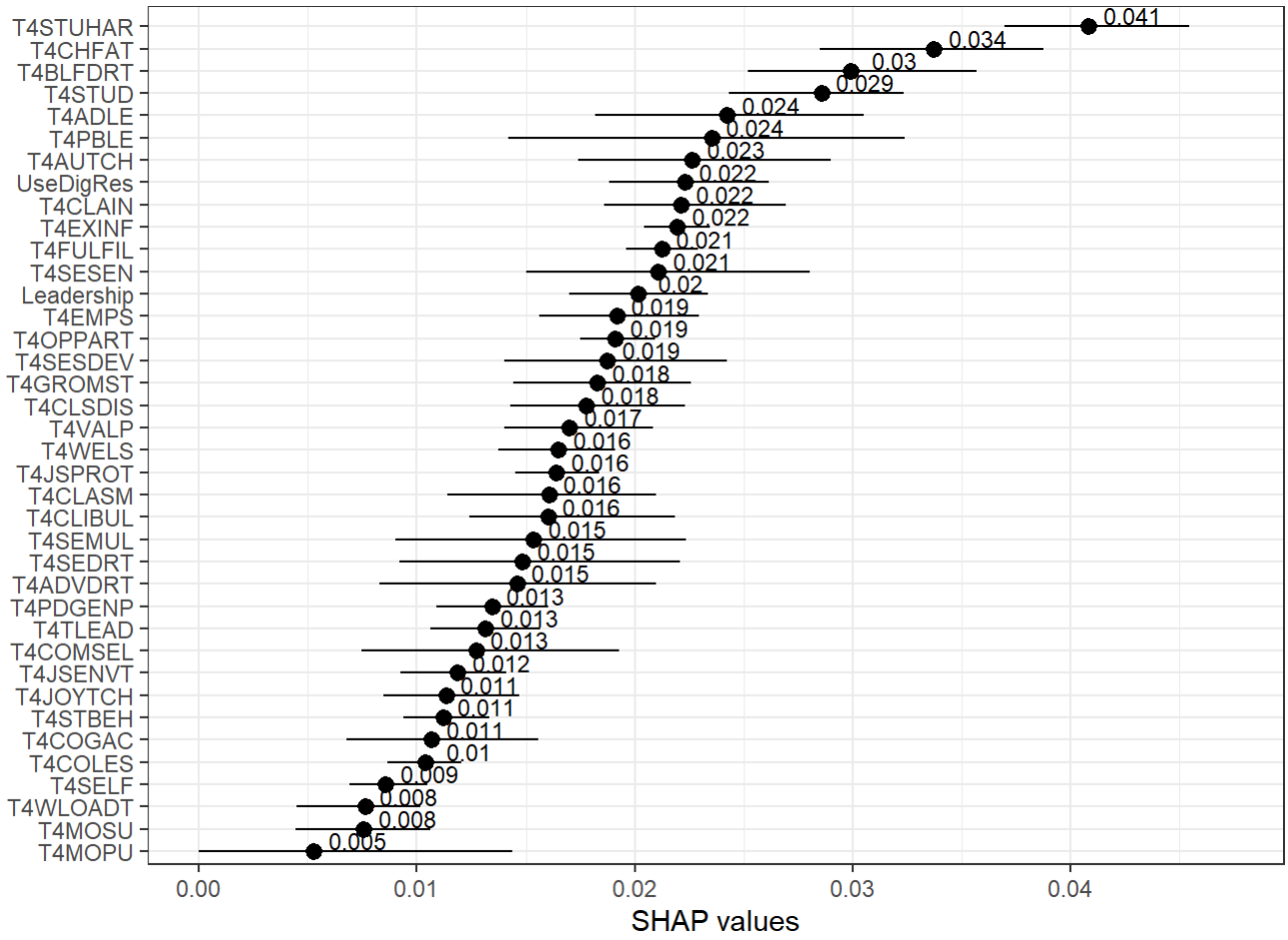

T4EXINF
T4COLES
T4SELF
T4SEMUL
T4GROMST
T4SESEN
T4COMSEL
T4SEDRT
T4BLFDRT
T4ADVDRT
T4CLAIN
T4COGAC
T4CLASM
T4PDGENP
T4EMPS
T4CLSDIS
T4ADLE
T4PBLE
T4AUTCH
T4FULFIL
T4OPPART
T4SESDEV
T4TLEAD
T4STUD
T4CLIBUL
T4STUHAR
T4CHFAT
T4MOPU
T4MOSU
T4WELS
T4WLOADT
T4STBEH
T4JSENVT
T4JSPROT
T4VALP
T4JOYTCH
UseDigRes
Leadership
split value
data
-4
-2
0
2
Value

Results

Variable selection and variable importance

```
shap_results$phis %>%
  lapply( function(x) colMeans(abs(x)) ) %>%
  lapply( function(x) data.frame(m   =mean(x),
                                 q10=quantile(x,0.10) ,
                                 q90=quantile(x,0.90) ) ) %>%
  bind_rows(.id="Var") -> phi_CI

phi_CI %>% arrange(m) %>%
  mutate(Var=factor(Var,unique(Var)) ) %>%
  ggplot(aes(x=Var,y=m,ymin=q10,ymax=q90)) +
    geom_point()+
    geom_text(aes(label=round(m,3)),position=position_nudge(0.48,0.002),size=3 )+
    geom_pointrange()+
    theme_bw() +
    coord_flip(xlim=c(1,38.3))+
    labs(x="",y="SHAP values")
```



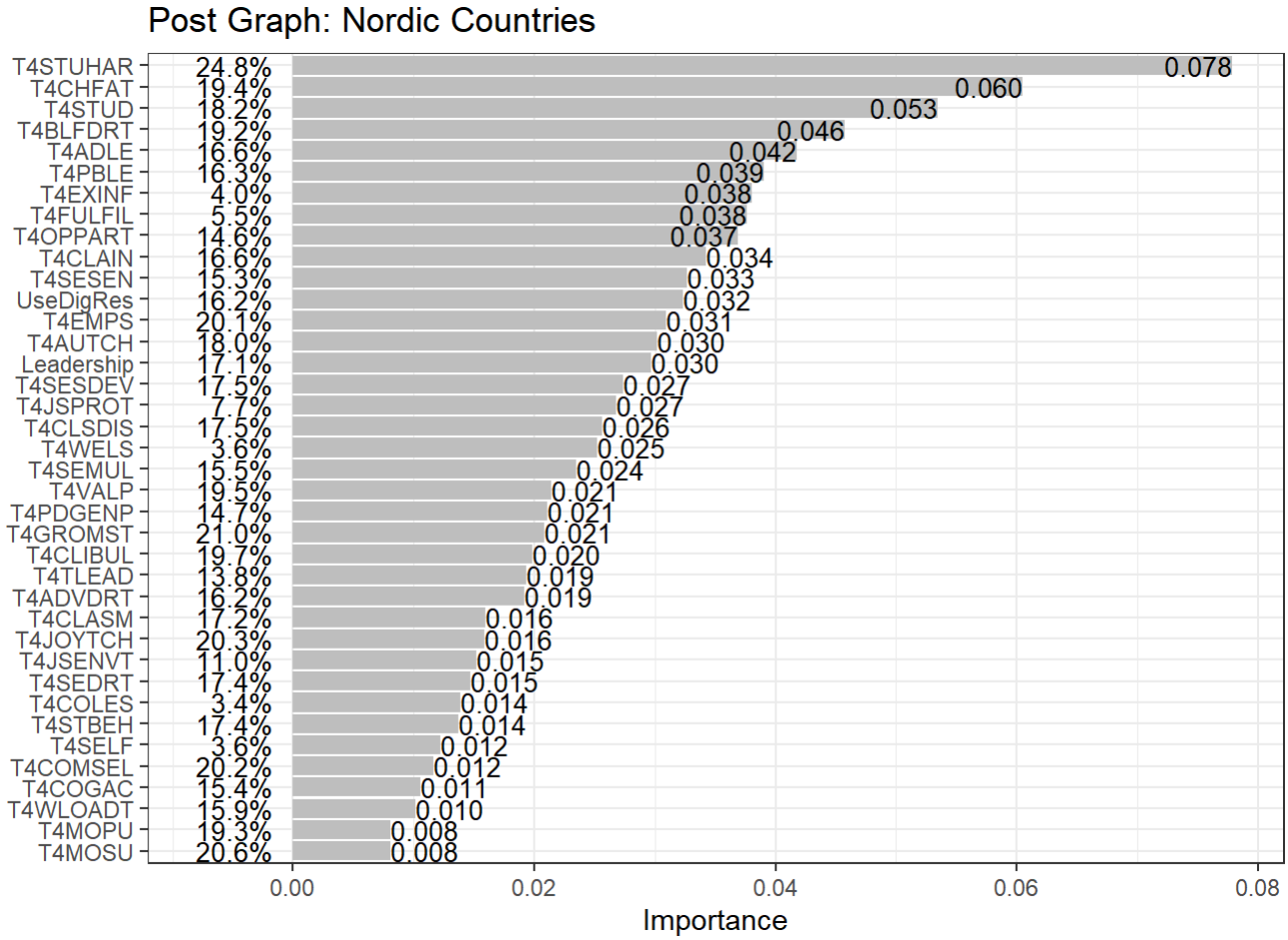

```
ggsave(file="customshapresults.png",height=14,width=14,units="cm")
```

Plot separate SHAP plots across the nordics and non-nordics.

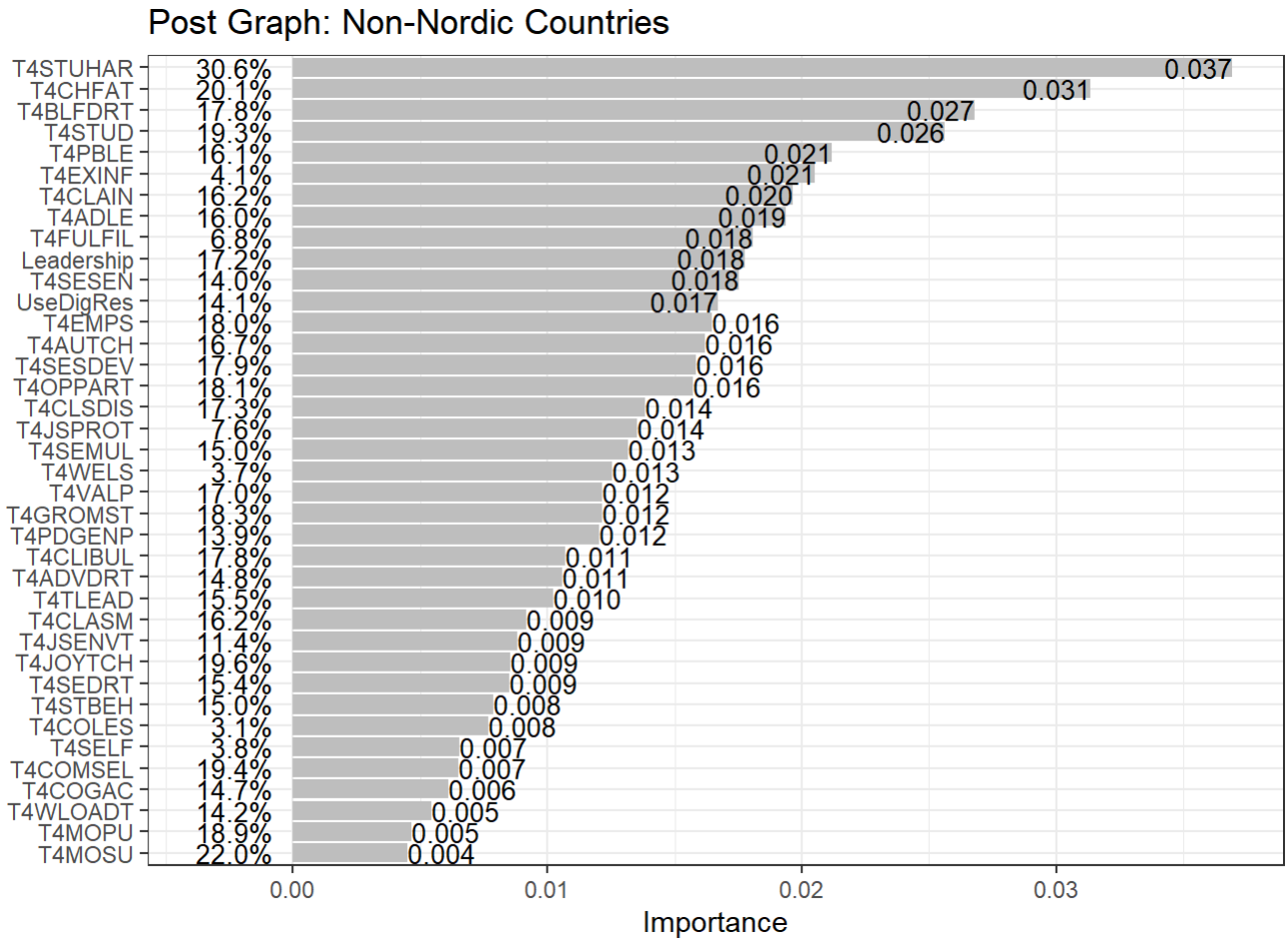
```
# 1. Duplicate your main SHAP object for the TRUE group
shap_nordic_true <- shap_results
# Filter the SHAP values (phis) and the row predictions (fx)
shap_nordic_true$phis <- lapply(shap_results$phis, function(x) x[Test$Nordic==1,, drop = FALS
E])
shap_nordic_true$newdata <- shap_nordic_true$newdata[Test$Nordic==1,, drop = FALSE]
shap_nordic_true$fx   <- shap_results$fx[Test$Nordic == 1]

# 2. Duplicate your main SHAP object for the FALSE group
shap_nordic_false <- shap_results
shap_nordic_false$phis <- lapply(shap_results$phis, function(x) x[Test$Nordic==0,, drop = FAL
SE])
shap_nordic_false$newdata <- shap_nordic_false$newdata[Test$Nordic==0,, drop = FALSE]
shap_nordic_false$fx   <- shap_results$fx[Test$Nordic == 0]

plot(shap_nordic_true,  average = "post", title= "Post Graph: Nordic Countries")
```

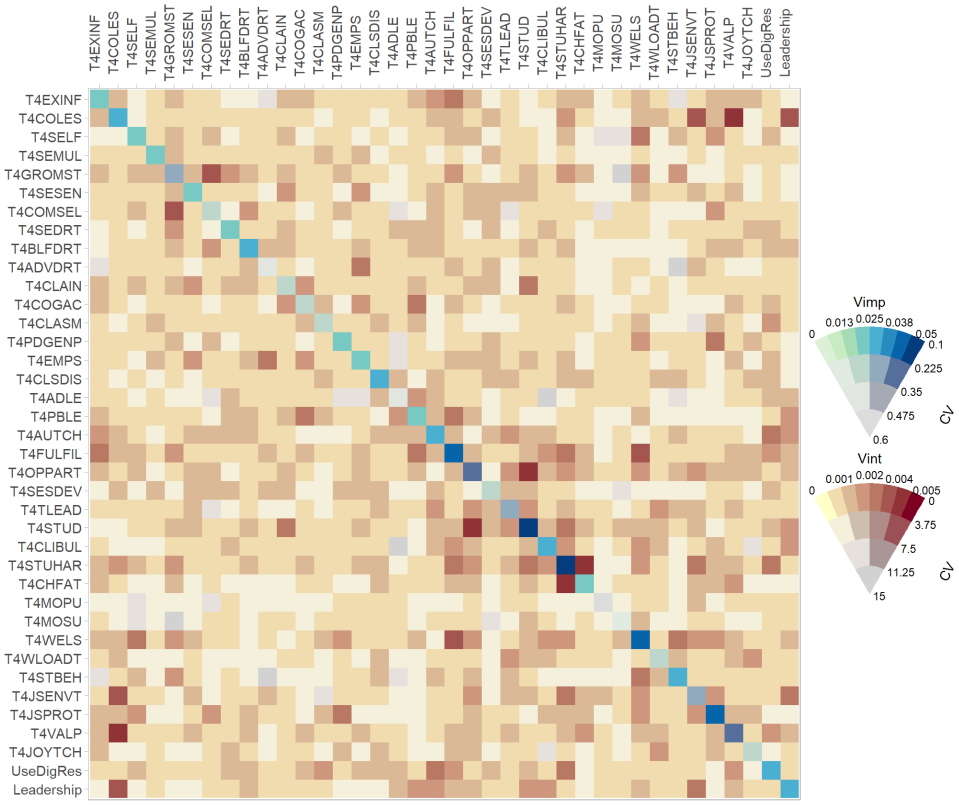



```
plot(shap_nordic_false, average = "post", title= "Post Graph: Non-Nordic Countries")
```



Interaction investigator

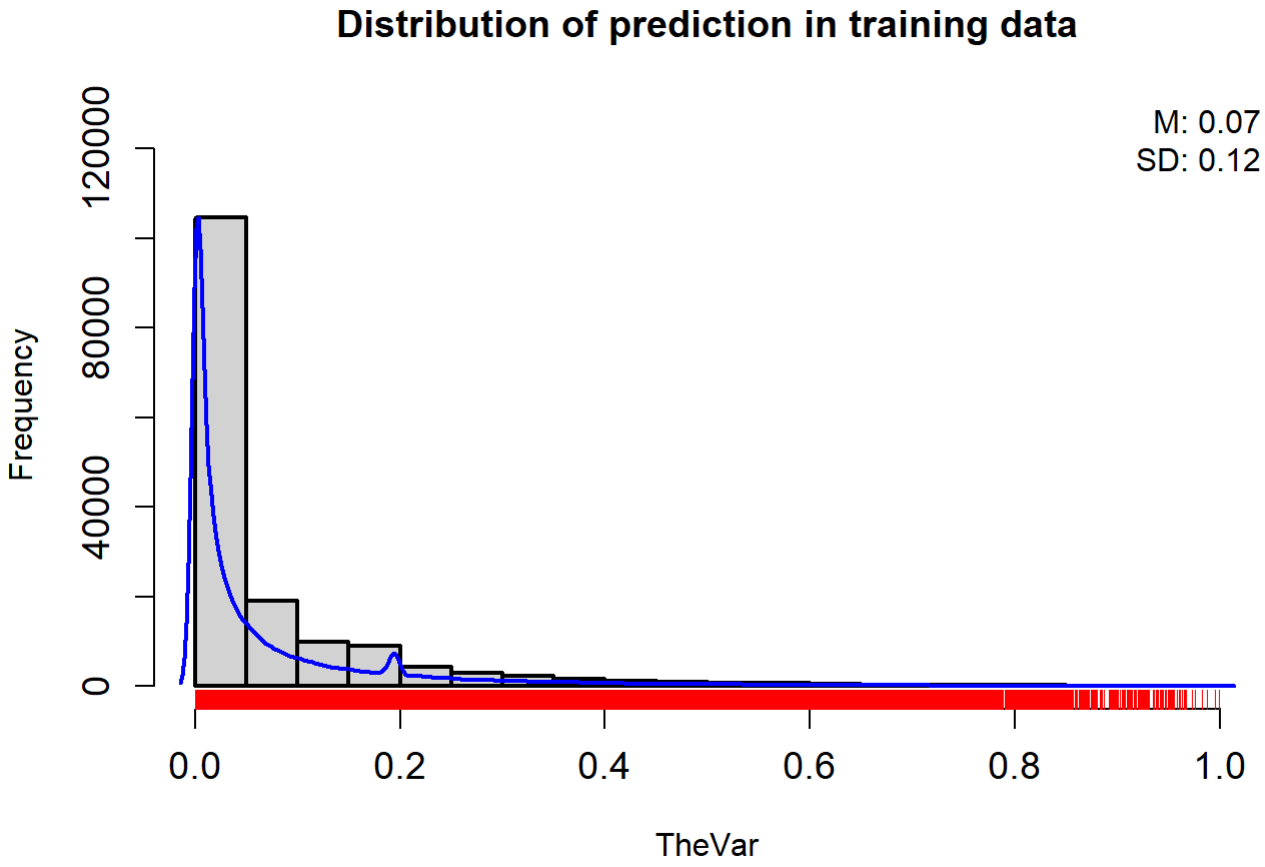
```
vsupMat <- viviBartMatrix(tree_data,
                          type = 'vsup',
                          metric = 'propMean',
                          metricError = "CV")
viviBartPlot(vsupMat,
             max_desat = 1,
             pow_desat = 0.6,
             max_light = 0.6,
             pow_light = 1,angle=90,
             label = 'CV')
```



distribution of predicted scores

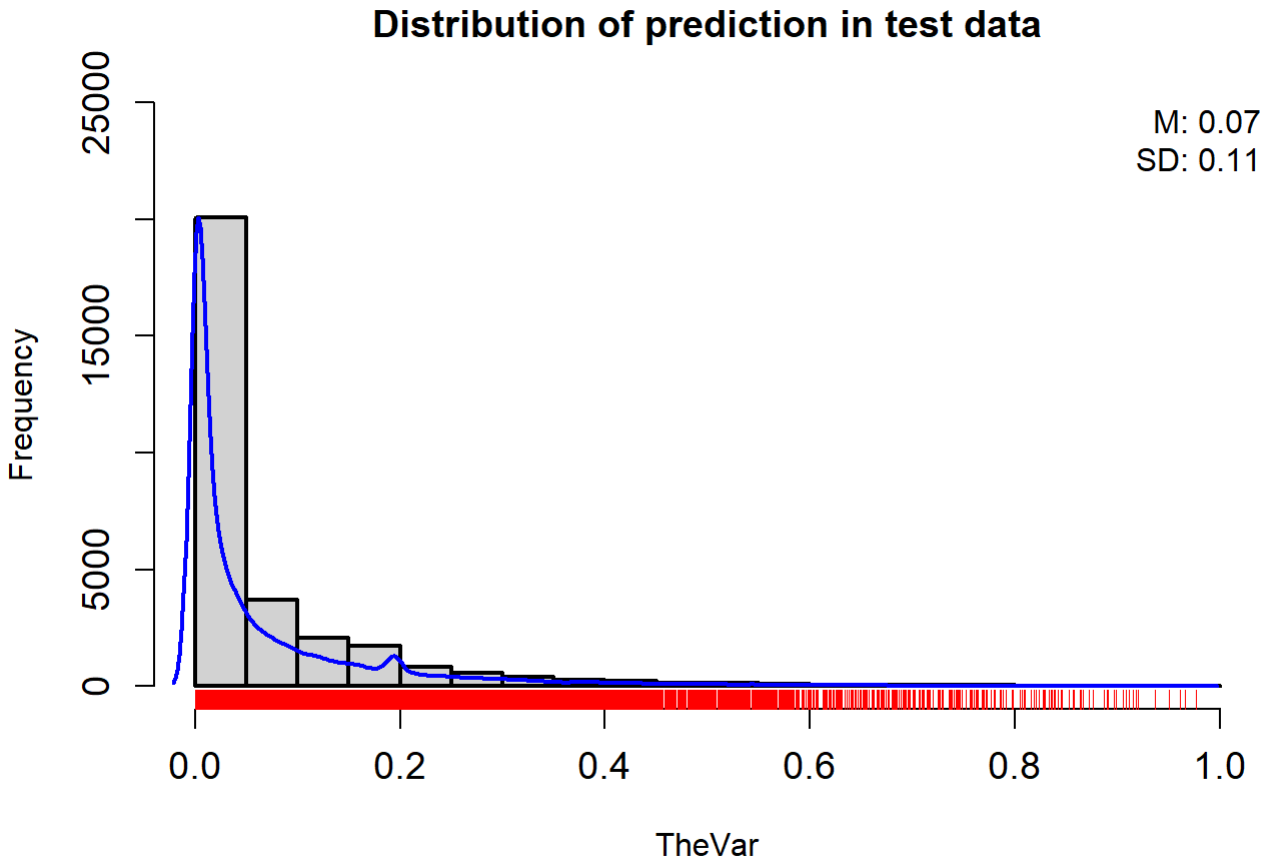
```
hist.rug(apply(predictedTrain,2,mean),main="Distribution of prediction in training data")
```

```
##    vars      n mean   sd median trimmed  mad min max range skew kurtosis se
## X1    1 158831 0.07 0.12   0.02    0.04 0.03   0   1     1 2.95    10.56  0
```




```
hist.rug(apply(predictedTest ,2,mean),main="Distribution of prediction in test data")
```

```
##    vars     n mean   sd median trimmed  mad min  max range skew kurtosis se
## X1    1 30515 0.07 0.11   0.02    0.04 0.03   0 0.98  0.98 2.94       11  0
```




```
# save(BTGINTT4, Test,predictedTrain,predictedTest,shap_results,
#      file="MainModelwithPredicted.rData")
```

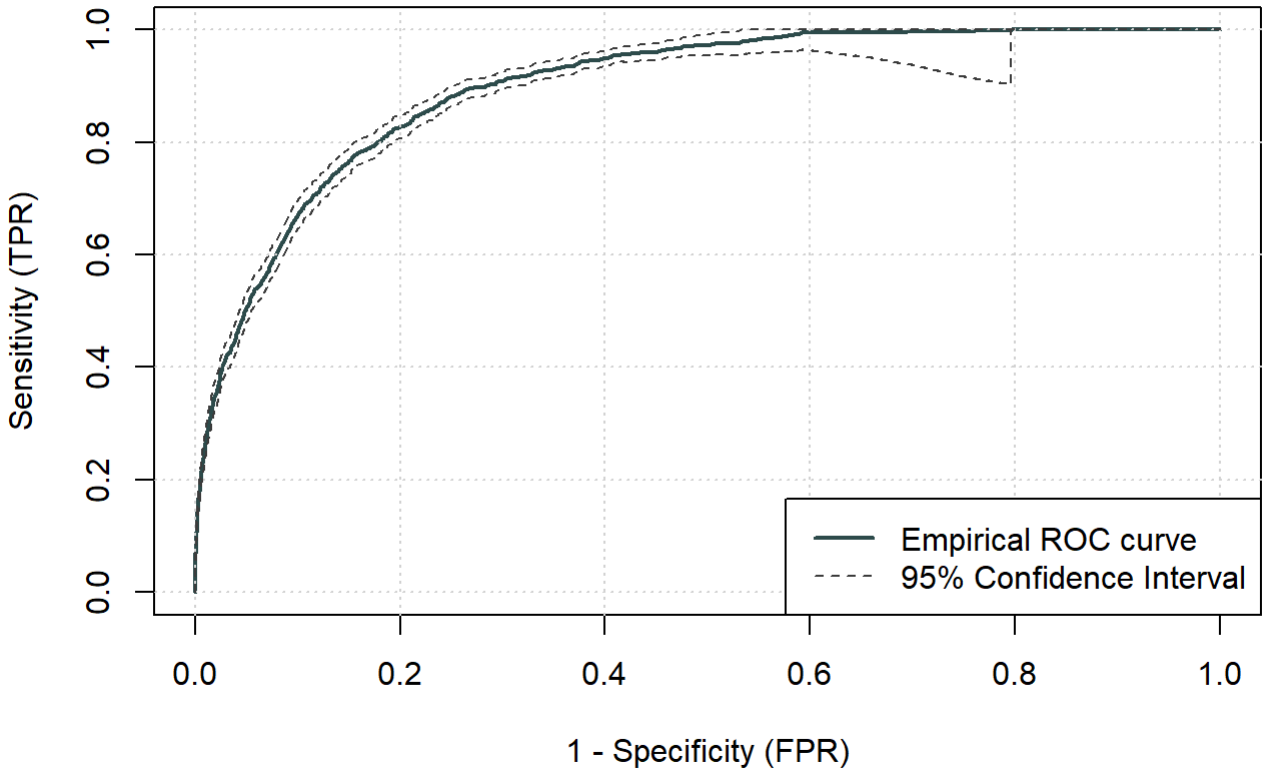
Print Prediction Tables using test data

```
str( ROCIT_Nordic   )
```

```
## List of 6
##  $ method                   : chr "empirical"
##  $ AUC                      : num 0.898
##  $ Cutoff                   : num [1:30078] Inf 0.976 0.966 0.966 0.961 ...
##  $ TPR/Sensitivity/Recall   : num [1:30078] 0 0.000759 0.001519 0.002278 0.003037 ...
##  $ FPR/1-Specificity        : num [1:30078] 0 0 0 0 0 0 0 0 0 0 ...
##  $ optimal Youden Index point: Named num [1:4] 0.6332 0.2468 0.88 0.0737
##   ..- attr(*, "names")= chr [1:4] "value" "FPR" "TPR" "cutoff"
```

```
ciROC(ROCit_obj, method = "binomial") %>% plot(legend=TRUE)
```

```
## Warning in regularize.values(x, y, ties, missing(ties), na.rm = na.rm): collapsing to uniq
ue 'x' values
```




```
ciAUC(ROCit_obj, delong = F)
```

```
##
##    estimated AUC : 0.897927175715377
##    AUC estimation method : empirical
##
##    CI of AUC
##    confidence level = 95%
##    lower = 0.886631104581589     upper = 0.909223246849166
```

Tables for classification

```
BTGINTT4 %>%
  transmute(CNTRY,predicted=apply(predictedTrain,2,mean)>=ROCIT_NordicTr$optimal[4]) %>%
  table
```

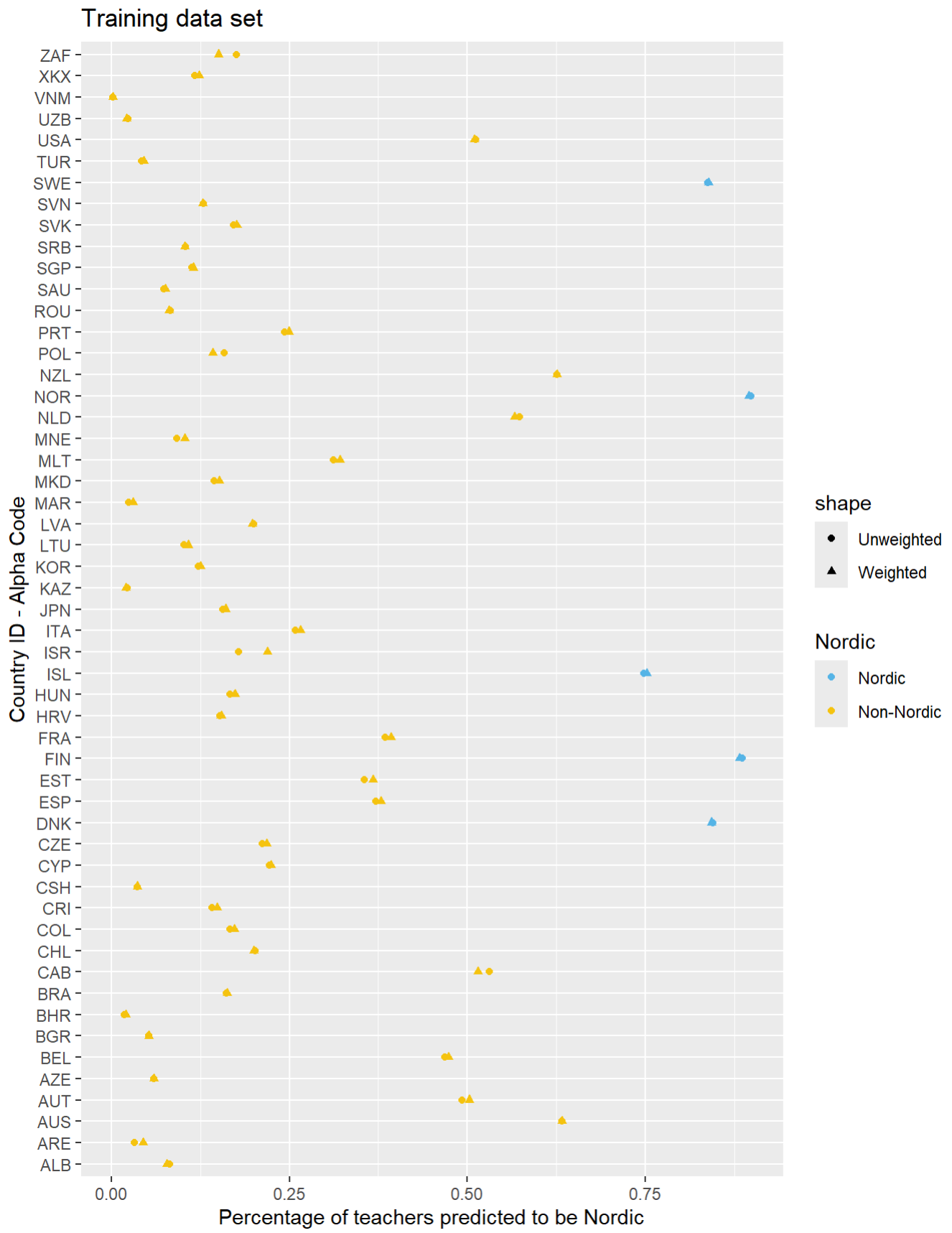
```
##      predicted
## CNTRY FALSE TRUE
##   ALB  3330  295
##   ARE  3804  126
##   AUS   999 1721
##   AUT  1970 1914
##   AZE  3640  232
##   BEL  2120 1871
##   BGR  3075  169
##   BHR  2739   52
##   BRA  2263  436
##   CAB   765  865
##   CHL  1663  421
##   COL  2244  450
##   CRI  2334  384
##   CSH  3532  133
##   CYP  1249  357
##   CZE  3143  848
##   DNK   346 1886
##   ESP  2509 1484
##   EST  1933 1064
##   FIN   437 3414
##   FRA  1590  995
##   HRV  2414  433
##   HUN  2905  580
##   ISL   322  954
##   ISR  2376  517
##   ITA  2583  900
##   JPN  2694  498
##   KAZ  3904   89
##   KOR  2445  339
##   LTU  3404  386
##   LVA  2943  735
##   MAR  3899   97
##   MKD  3071  517
##   MLT  1382  627
##   MNE  1101  112
##   NLD   486  652
##   NOR   139 1234
##   NZL   402  672
##   POL  2865  538
##   PRT  2378  764
##   ROU  3190  288
##   SAU  2753  219
##   SGP  2674  341
##   SRB  3220  372
##   SVK  3116  647
##   SVN  2217  330
##   SWE   447 2312
##   TUR  3808  170
##   USA   873  915
##   UZB  3880   92
##   VNM  3890    8
##   XKX  2606  346
##   ZAF  2439  519
```

```
Test %>%
  transmute(CNTRY,predicted=apply(predictedTest,2,mean)>=ROCIT_NordicTr$optimal[4]) %>%
  table
```

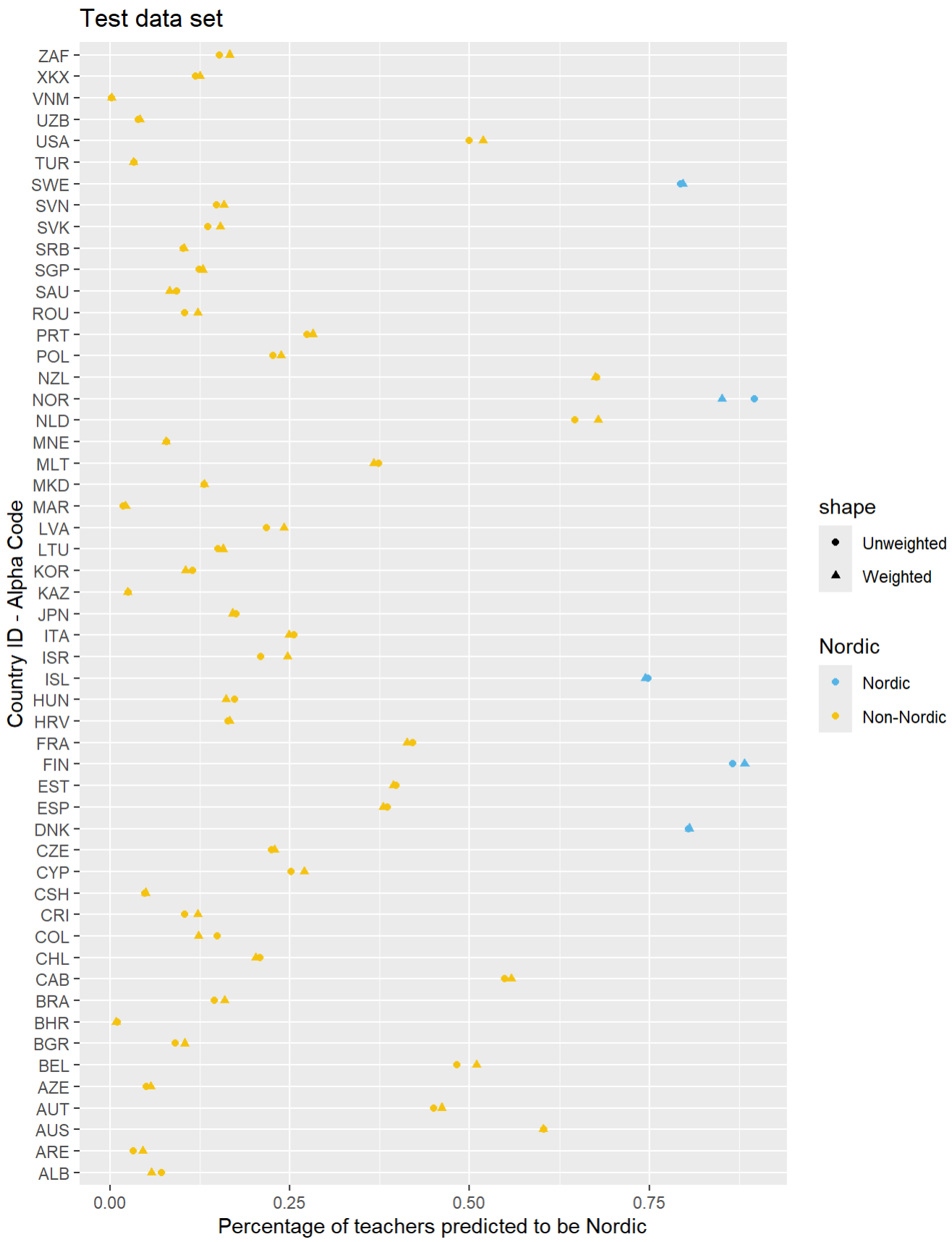
```
##      predicted
## CNTRY FALSE TRUE
##   ALB   375   29
##   ARE   425   14
##   AUS   125  190
##   AUT   248  203
##   AZE   413   22
##   BEL   613  571
##   BGR   350   35
##   BHR   310    3
##   BRA   265   45
##   CAB    87  106
##   CHL   186   49
##   COL   269   47
##   CRI   276   32
##   CSH   393   20
##   CYP   140   47
##   CZE  1843  535
##   DNK    50  206
##   ESP  3134 1971
##   EST   206  136
##   FIN    60  389
##   FRA   169  123
##   HRV   264   52
##   HUN   335   70
##   ISL    38  113
##   ISR   264   70
##   ITA   291  100
##   JPN   302   64
##   KAZ  3282   85
##   KOR   278   36
##   LTU   368   65
##   LVA   321   89
##   MAR  2078   38
##   MKD   351   53
##   MLT   159   95
##   MNE   130   11
##   NLD    47   86
##   NOR    16  139
##   NZL    42   88
##   POL   294   86
##   PRT   268  101
##   ROU   354   41
##   SAU   303   31
##   SGP   298   42
##   SRB   373   42
##   SVK   369   58
##   SVN   242   42
##   SWE    63  243
##   TUR   922   32
##   USA   100  100
##   UZB   441   18
##   VNM   449    1
##   XKX   290   39
##   ZAF   291   52
```

Comparison of weighted and unweighted results

```
BTGINTT4 %>%
  mutate(predicted = apply(predictedTrain,2,mean),
         Nordic=factor(Nordic,1:0,c("Nordic","Non-Nordic")) ) %>%
  group_by(CNTRY,Nordic) %>%
  summarize(Unwg_pred=mean(predicted          >=ROCIT_NordicTr$optimal[4]),
            Wgt_pred =weighted.mean(predicted >=ROCIT_NordicTr$optimal[4],unlabelled(TCHWGT))
,
            .groups="drop") %>%
  ggplot(aes(y=CNTRY,color=Nordic))+
    geom_point(aes(x=Unwg_pred,shape="Unweighted"))+
    geom_point(aes(x=Wgt_pred,shape="Weighted"))+
    labs(x="Percentage of teachers predicted to be Nordic",title="Training data set")
```


Training data set
Country ID - Alpha Code
ZAF
XKX
VNM
UZB
USA
TUR
SWE
SVN
SVK
SRB
SGP
SAU
ROU
PRT
POL
NZL
NOR
NLD
MNE
MLT
MKD
MAR
LVA
LTU
KOR
KAZ
JPN
ITA
ISR
ISL
HUN
HRV
FRA
FIN
EST
ESP
DNK
CZE
CYP
CSH
CRI
COL
CHL
CAB
BRA
BHR
BGR
BEL
AZE
AUT
AUS
ARE
ALB
0.00
0.25
0.50
0.75
Percentage of teachers predicted to be Nordic
shape
Unweighted
Weighted
Nordic
Nordic
Non-Nordic

```
Test %>%
  mutate(predicted = apply(predictedTest,2,mean),
         Nordic=factor(Nordic,1:0,c("Nordic","Non-Nordic")) ) %>%
  group_by(CNTRY,Nordic) %>%
  summarize(Unwg_pred=mean(predicted>=ROCIT_NordicTr$optimal[4]),
            Wgt_pred =weighted.mean(predicted>=ROCIT_NordicTr$optimal[4],unlabelled(TCHWGT))
,
            .groups="drop") %>%
  ggplot(aes(y=CNTRY,color=Nordic))+
    geom_point(aes(x=Unwg_pred,shape="Unweighted"))+
    geom_point(aes(x=Wgt_pred,shape="Weighted"))+
    labs(x="Percentage of teachers predicted to be Nordic",title="Test data set")
```


Test data set
Country ID - Alpha Code
ZAF
XKX
VNM
UZB
USA
TUR
SWE
SVN
SVK
SRB
SGP
SAU
ROU
PRT
POL
NZL
NOR
NLD
MNE
MLT
MKD
MAR
LVA
LTU
KOR
KAZ
JPN
ITA
ISR
ISL
HUN
HRV
FRA
FIN
EST
ESP
DNK
CZE
CYP
CSH
CRI
COL
CHL
CAB
BRA
BHR
BGR
BEL
AZE
AUT
AUS
ARE
ALB
0.00
0.25
0.50
0.75
Percentage of teachers predicted to be Nordic
shape
Unweighted
Weighted
Nordic
Nordic
Non-Nordic

Accuracy in Test data

```
Test %>% transmute(CNTRY,predicted=ifelse(apply(predictedTest,2,mean) >0.086,"Nordic","Non")
) %>%
  table %>% as.data.frame %>%
  mutate(CNTRY=ifelse(CNTRY %in% c("DNK","NOR","SWE","FIN","ISL"),"Nordic","Non") ) %>%
  group_by(predicted,CNTRY) %>%
  summarize(Freq=sum(Freq) , .groups="drop") %>%
  with(
    data.frame(TruePos =Freq[predicted=="Nordic" & CNTRY=="Nordic"]/sum(Freq[CNTRY=="Nordi
c"]),
               FalsePos=Freq[predicted!="Nordic" & CNTRY=="Nordic"]/sum(Freq[CNTRY=="Nordi
c"]),
               TrueNeg =Freq[predicted!="Nordic" & CNTRY!="Nordic"]/sum(Freq[CNTRY!="Nordi
c"]),
               FalseNeg=Freq[predicted=="Nordic" & CNTRY!="Nordic"]/sum(Freq[CNTRY!="Nordi
c"])
               )
  )
```

```
##     TruePos  FalsePos   TrueNeg  FalseNeg
## 1 0.8504176 0.1495824 0.7796767 0.2203233
```

Session info

```
sessionInfo()
```

```
## R version 4.6.1 (2026-06-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows Server 2022 x64 (build 26100)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] C
##
## time zone: Europe/Oslo
## tzcode source: internal
##
## attached base packages:
## [1] splines   grid      stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] ggridges_0.5.7      future.apply_1.20.2 furrr_0.4.0         future_1.75.0       data.
table_1.18.4   bartXViz_1.0.11     SuperLearner_2.0-42 nnls_1.6            gam_1.22-7
## [10] foreach_1.5.2       labelled_2.16.0     tinytex_0.60        arm_1.15-3          lme4_
2.0-6          MASS_7.3-66         haven_2.5.5         posterior_1.7.0     ROCit_2.1.2
## [19] coda_0.19-4.1       survey_4.5          survival_3.8-9      Matrix_1.7-5        lubri
date_1.9.5     forcats_1.0.1       stringr_1.6.0       dplyr_1.2.1         purrr_1.2.2
## [28] readr_2.2.0         tidyr_1.3.2         tibble_3.3.1        ggplot2_4.0.3       tidyv
erse_2.0.0     bartMan_0.2.1       dbarts_0.9-33
##
## loaded via a namespace (and not attached):
##   [1] bitops_1.1-0            polyclip_1.10-7         gggenes_0.7.0           lifecycle_1.
0.5         Rdpack_2.6.6            rstatix_1.1.0           rrapply_1.2.8           globals_
0.19.1
##   [9] lattice_0.22-9          backports_1.5.1         magrittr_2.0.5          sass_0.4.10
rmarkdown_2.31          jquerylib_0.1.4         yaml_2.3.12             otel_0.2.0
##  [17] doRNG_1.8.6.3           BART_2.9.10             cowplot_1.2.0           DBI_1.3.0
minqa_1.2.8             RColorBrewer_1.1-3      DendSer_1.0.3           abind_1.4-8
##  [25] ggraph_2.2.2            itertools_0.1-3         tensorA_0.36.2.1        tweenr_2.0.3
gdtools_0.5.1           seriation_1.5.8         ggrepel_0.9.8           listenv_1.0.0
##  [33] missForest_1.6.1        parallelly_1.48.0       codetools_0.2-20        ggforce_0.5.
0           tidyselect_1.2.1        farver_2.1.2            viridis_0.6.5           TSP_1.2.7
##  [41] jsonlite_2.0.0          tidygraph_1.3.1         Formula_1.2-6           iterators_1.
0.14        systemfonts_1.3.2       tools_4.6.1             ggnewscale_0.5.2        ragg_1.5.
2
##  [49] Rcpp_1.1.2              glue_1.8.1              mnormt_2.1.2            gridExtra_2.
3.1         xfun_0.60               ranger_0.18.0           mgcv_1.9-4              distribut
ional_0.8.1
##  [57] ca_0.71.1               withr_3.0.3             fastmap_1.2.0           mitools_2.4
boot_1.3-32             caTools_1.18.4          digest_0.6.39           gclus_1.3.3
##  [65] timechange_0.4.0        R6_2.6.1                textshaping_1.0.5       colorspace_
2.1-3        gtools_3.9.5            see_0.14.1              generics_0.1.4          misty_0.
8.3
##  [73] fontLiberation_0.1.0    graphlayouts_1.2.5      htmlwidgets_1.6.4       pkgconfig_2.
0.3         rJava_1.0-18            gtable_0.3.6            registry_0.5-1          S7_0.2.2
##  [81] htmltools_0.5.9         bartMachineJARs_1.2.2   fontBitstreamVera_0.1.1 carData_3.0-
6           scales_1.4.0            reformulas_0.4.4        knitr_1.51              rstudioap
i_0.19.0
##  [89] tzdb_0.5.0              reshape2_1.4.5          checkmate_2.3.4         nlme_3.1-169
```

```
nloptr_2.2.1            cachem_1.1.0            KernSmooth_2.23-26      parallel_4.6.1
##  [97] pillar_1.11.1          vctrs_0.7.3            gplots_3.3.0           randomForest
_4.7-1.2    ggpubr_1.0.0           ggfittext_0.10.4       car_3.1-5              cluster_
2.1.8.2
## [105] evaluate_1.0.5         cli_3.6.6              compiler_4.6.1         rlang_1.3.0
rngtools_1.5.2          ggsignif_0.6.4         labeling_0.4.3         plyr_1.8.9
## [113] ggiraph_0.9.6          stringi_1.8.9          psych_2.6.5            viridisLite_
0.4.3       fontquiver_0.2.1       hms_1.1.4              patchwork_1.3.2        rbibutils
_2.4.1
## [121] igraph_2.3.3           broom_1.0.13           memoise_2.0.1          bslib_0.12.0
bartMachine_1.4.2
```